

A REPORT
ON
**MODEL DEPLOYMENT USING FASTAPI AND
DOCKER**

Submitted by

Ms. Shaziya Banu

IS-2026-9761

INTERNSPARK

ARTIFICIAL INTELLIGENCE INTERNSHIP

Jul-2026

1. Introduction

Machine Learning models are most valuable when they can be integrated into real-world applications and accessed by users or systems. Model deployment is the process of making a trained machine learning model available for prediction through an Application Programming Interface (API).

In this project, the Customer Churn Prediction model developed in Task 1 was deployed using FastAPI. FastAPI is a modern Python framework used for building high-performance APIs. The deployed model can receive requests and return churn predictions through REST endpoints.

To ensure portability and reproducibility, a Docker configuration was also prepared for containerized deployment.

2. Objective

The objectives of this project are:

- Deploy a trained machine learning model as a REST API.
- Create endpoints for model inference.
- Test API functionality using FastAPI.
- Access automatic API documentation through Swagger UI.
- Prepare the application for containerized deployment using Docker.

3. Project Description

The Customer Churn Prediction model created in Task 1 was selected for deployment.

Model Characteristics

- Model Type: Logistic Regression
- Problem Type: Binary Classification
- Target Variable: Churn
- Input Features: 19
- Prediction Output: Churn / No Churn

Prediction Labels

- Churn = 1 (Customer likely to churn)
- Churn = 0 (Customer likely to stay)

4. Model Loading

The trained Logistic Regression model was saved using the Pickle library and loaded into the FastAPI application during startup.

Steps Performed:

- Imported the Pickle library.
- Loaded the saved churn_model.pkl file.

- Initialized the model during application startup.

Observation:

The model was successfully loaded and made available for prediction requests.

5. API Development

FastAPI was used to create REST endpoints for model access.

5.1 Home Endpoint

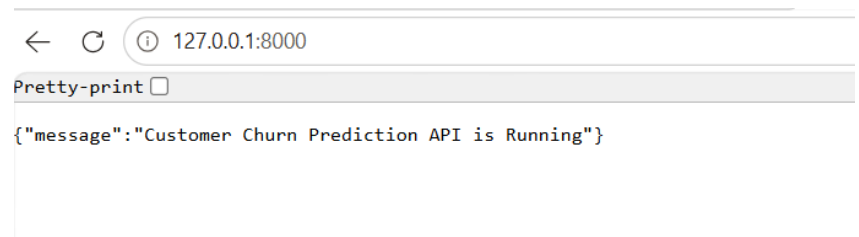
A root endpoint was created to verify that the API server is running successfully.

Endpoint:

GET /

Response:

```
{  
  "message": "Customer Churn  
Prediction API is Running"  
}
```



Observation:

The endpoint successfully confirms that the API service is active.

5.2 Prediction Endpoint

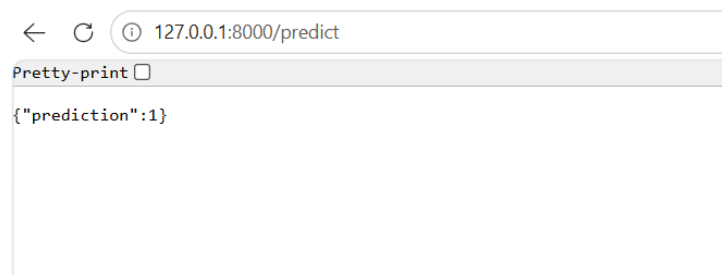
A prediction endpoint was created to generate churn predictions using customer input data.

Endpoint:

GET /predict

Response Example:

```
{  
  "prediction": 1  
}
```



Observation:

The API successfully loads the trained model and returns churn predictions.

6. API Testing

The API was executed locally using Uvicorn.

Server Command:

```
uvicorn main:app --reload
```

Local Server URL:

<http://127.0.0.1:8000>

Testing Results:

- API server started successfully.
- Home endpoint returned expected response.
- Prediction endpoint generated valid churn predictions.
- No runtime errors were encountered during testing.

7. Swagger Documentation

FastAPI automatically generates interactive API documentation using Swagger UI.

Swagger URL:

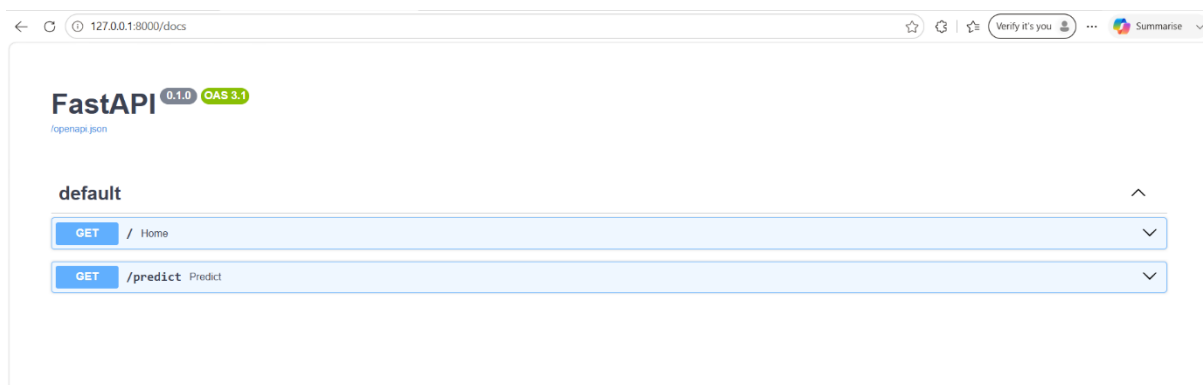
<http://127.0.0.1:8000/docs>

Benefits:

- Interactive endpoint testing.
- Automatic API documentation.
- Easy validation of API requests and responses.

Observation:

Swagger documentation was generated successfully and displayed all available endpoints.



8. Docker Configuration

A Dockerfile was created to package the application and its dependencies into a portable container.

Docker allows the application to run consistently across different systems without requiring manual dependency installation.

Docker Configuration Includes:

- Python runtime environment.
- FastAPI framework.
- Required machine learning libraries.
- API startup commands.

Observation:

The Dockerfile prepares the application for future containerized deployment.

9. Results

The model deployment process was completed successfully.

Results Obtained:

- FastAPI application created successfully.
- Logistic Regression model loaded correctly.
- REST API endpoints implemented.
- Churn predictions generated successfully.
- Swagger documentation generated automatically.
- Docker configuration prepared successfully.

Sample Prediction Output:

```
{  
  "prediction": 1  
}
```

Interpretation:

The model predicts that the customer is likely to churn.

10. Conclusion

In this project, the Customer Churn Prediction model developed in Task 1 was successfully deployed using FastAPI. REST API endpoints were created to provide real-time access to model predictions. Swagger documentation enabled easy testing and validation of the API.

Additionally, Docker support was prepared to simplify future deployment across different environments. This project demonstrates the complete machine learning workflow from model development to deployment and accessibility through web services.

Technologies Used

- Python 3.11
- FastAPI
- Uvicorn
- NumPy
- Scikit-Learn
- Pickle
- Docker
- VS Code

- Anaconda

References

1. FastAPI Documentation
2. Uvicorn Documentation
3. Scikit-Learn Documentation
4. Docker Documentation
5. IBM Telco Customer Churn Dataset